



PRÁCTICA 04: ESCRITURA DE FICHEROS

1. Crea un programa que escriba en un fichero los 10 primeros números de Fibonacci, cada uno separado por un espacio del siguiente.

2. Crea un programa que pida al usuario que escriba pequeños textos por pantalla. Cada vez que pulse intro se volverá a pedir otro texto, a no ser que no haya escrito nada más, en cuyo caso dejará de pedir que el usuario escriba. El programa escribirá cada texto que el usuario ha escrito en una línea de un fichero con extensión .txt.

3. Crea un programa que lea el contenido del fichero numeros.txt, y luego escriba los números pares en un fichero llamado pares.txt y los números impares en un fichero llamado impares.txt.

4. Crea un programa que guarde la información de una lista de objetos de la clase Libro en un fichero llamado "libros.csv". La clase Libro incluirá los siguientes atributos:
 - a. Título
 - b. Autor
 - c. Año publicación
 - d. ISBN

La información de cada libro se almacenará en una línea distinta del fichero. Y el formato de la línea será el siguiente:

Título;Autor;Año publicación;ISBN

5. Crea un programa que haga copias de cualquier tipo de fichero. El usuario nos proporcionará por consola la ruta del fichero de origen y la ruta del fichero destino.

6. Crea un programa que nos recomendará juegos de mesa en función de las necesidades de nuestro grupo de jugadores.

En primer lugar, el programa leerá el fichero “juegos.csv”, que almacena en cada línea las características de un juego, correspondiente con la siguiente estructura:

Nombre;EdadMínima;MinJugadores;MaxJugadores;Duración

Después nos pedirá la edad de nuestro jugador más joven, cuántos jugadores somos y cuánto tiempo tenemos para jugar.

En función de esos datos, nos devolverá por pantalla los nombres de los juegos del listado original a los que podríamos jugar.

Finalmente nos preguntará si queremos guardar la recomendación en un fichero, si decimos que sí, se guardará en un fichero “juegosRecomendados.csv” donde cada fila contendrá los datos de un juego, de forma similar al fichero “juegos.csv”.

7. Construye una aplicación en Java que organice el almacenamiento binario de datos numéricos en el sistema de archivos. El programa debe comprobar si existe una carpeta específica destinada a guardar datos; si no existe, debe crearla. A continuación, generará un archivo binario dentro de dicha carpeta y escribirá en él una secuencia de bytes con los valores de la tabla de multiplicar del número 5 (del 5 al 50). En caso de que el archivo ya existiera de ejecuciones previas, el programa deberá añadir la nueva serie de números al final del fichero existente sin borrar los datos anteriores. El tratamiento de las posibles anomalías en la entrada y salida de datos debe gestionarse de forma explícita.

8. Desarrolla un programa en Java que audite el contenido de un directorio del proyecto y vuelque un resumen detallado en un fichero de texto. La aplicación debe consultar el contenido del directorio raíz actual e inspeccionar los nombres de todos los elementos presentes. Posteriormente, debe crear un archivo de texto formateado donde escriba línea a línea cada nombre precedido por un número correlativo y su tipo (si es un fichero o un directorio). Al finalizar el listado, se debe incluir una línea de cierre que indique la cantidad total de elementos contabilizados. Si la carpeta de informe ya contiene un archivo con el mismo nombre, su contenido previo debe reemplazarse por completo.

9. Crea una herramienta en Java para realizar duplicaciones de seguridad de ficheros binarios copiando sus datos en bloques de memoria de tamaño fijo. El programa debe verificar en primer lugar la presencia del archivo de origen; si este no existe, debe lanzar y capturar una excepción propia personalizada denominada `ExcepcionFicheroOrigenNoEncontrado`. Si la carpeta de destino no existe en el sistema, la aplicación creará toda la estructura de directorios necesaria antes de iniciar la copia. La copia se realizará leyendo el archivo de origen en tramos de 1024 bytes e inscribiéndolos de forma continua en el fichero de destino, garantizando la liberación limpia de los recursos de entrada y salida empleados.

10. Construye una aplicación en Java que procese un archivo de texto plano mediante filtrado y transformación de sus cadenas de caracteres. El programa debe leer un archivo de entrada línea por línea, convertir todo el texto a mayúsculas, eliminar los espacios en blanco iniciales y finales, y escribir las líneas procesadas en un nuevo archivo de texto formateado. La lógica de lectura, transformación y escritura se debe implementar dentro de un método auxiliar que declare expresamente la propagación de excepciones de entrada/salida hacia el método principal. Para la escritura, se deben conectar secuencialmente flujos de bytes, convertidores de caracteres y escrituras con búfer, asegurando el vaciado de los datos y el cierre ordenado de todas las capas interconectadas